RESUMEN DE SENTENCIAS DML DE MYSQL

DML es el lenguaje de manipulación de datos en MySQL. Incluye sentencias que sirven para añadir nuevos registros a una tabla, actualizarlos cambiando sus valores, eliminar registros o seleccionar registros para guardarlos en un objeto *recordset* (un conjunto de registros, algo similar a un array de dos dimensiones).

1. Sentencia INSERT

Sirve para insertar, añadir nuevos registros a una tabla. Su sintaxis básica es:

```
INSERT INTO `tabla` (`campo1`, `campo2`,…)
            VALUES ( valor1,   valor2,…),
                        ……................................
                   ( valor1,   valor2,…);
```

Recordad que los valores que asignamos deben ir en la cláusula `VALUES` en el mismo orden que hayamos especificado para los campos en la cláusula `INTO`. Si queremos introducir al mismo tiempo varios registros, especificamos sus valores entre paréntesis separados mediante comas.

Nota: Recordad que la especificación de SQL obliga a delimitar los nombres de tablas y campos mediante comillas de tipo (`). También debemos finalizar la sentencia con punto y coma.

Por ejemplo:

```
INSERT INTO `EMPLEADOS`
       (`NUM_EMPLEADO`, `NOMBRE`, `PUESTO`, `JEFE`,
        `FECH_CONTRAT`, `SALARIO`, `COMISION`, `NUM_DEPT`)
VALUES (7940,'GERVASIO','EMPLEADO',7902,'1981-12-31',
        1200,NULL,10),
       (7950,'DOMITILIO','EMPLEADO',7902,'1981-10-26',
        1400,NULL,20);
```

2. Sentencia UPDATE

Sirve para actualizar el contenido de uno o varios campos de una tabla. Su sintaxis básica es la siguiente:

```
UPDATE `tabla`
SET `campo1` = valor1, `campo2` = valor2, ...
WHERE condición;
```

Aquí la *condición* funciona exactamente igual que en cualquier lenguaje de programación. Sirve para evaluar los valores que deben tener los registros que se van a ver afectados por la consulta de actualización. Sólo se actualizarán los registros que cumplan dicha condición. Por ejemplo:

```
UPDATE `EMPLEADOS`
SET `NOMBRE`='ATAULFO'
WHERE `NUM_EMPLEADO`=7950;
```

Frecuentemente se utilizan las sentencias `UPDATE` para realizar operaciones con los valores de los campos de una tabla. Por ejemplo, imaginémonos que queremos subir el sueldo a todos los empleados que trabajen como vendedores. Podríamos utilizar una expresión como la siguiente:

```
UPDATE `EMPLEADOS`
SET `SALARIO`=`SALARIO` * 1.10
WHERE `PUESTO`='VENDEDOR';
```

Nota: Tened cuidado con la cláusula `WHERE`. Si no es correcta, podéis modificar registros no deseados. Siempre es mejor probar siempre la cláusula `WHERE` con una sentencia `SELECT`.

3. Sentencia DELETE

Permite eliminar registros en una base de datos. Su sintaxis básica es:

```
DELETE FROM `tabla`
WHERE condición
```

Aquí adquiere especial importancia la condición de la cláusula `WHERE`. Si omitimos dicha cláusula, ¡se borrarán todos los registros de la tabla! Es mejor probar siempre nuestras sentencias `DELETE` como sentencias `SELECT` antes de ejecutarlas.

La siguiente consulta borra todos los empleados de la tabla `empleados` que tienen un número de empleado superior o igual a 7940:

```
DELETE FROM `empleados`
WHERE `NUM_EMPLEADO` >= 7940;
```

4. Sentencia SELECT

Esta sentencia devuelve un conjunto de registros (*recordset*) que contiene los campos y los registros especificados en sus diferentes cláusulas. Su sintaxis básica es:

```
SELECT `campo1`, `campo2`,…
FROM `tabla`;
```

Si quisiéramos mostrar todos los campos de la tabla, podemos utilizar el selector universal (*), de la siguiente forma:

```
SELECT * FROM `tabla`;
```

Por ejemplo, para mostrar los nombres y salarios de todos los empleados de mi empresa escribiría la siguiente sentencia SQL:

```
SELECT `NOMBRE`, `SALARIO`
FROM `EMPLEADOS`;
```

Si queremos filtrar el número de registros que devolverá la sentencia `SELECT`, tendremos que utilizar una cláusula `WHERE`. En dicha cláusula especificaremos una condición. Sólo los registros que cumplan dicha condición aparecerán en el resultado final.

Por ejemplo, para visualizar los empleados que cobran menos de 1000 euros, podríamos utilizar la siguiente sentencia:

```
SELECT `NOMBRE`, `SALARIO`
FROM `EMPLEADOS`
WHERE `SALARIO` < 1000;
```

También podemos ordenar los resultados de nuestra consulta. Para hacerlo, utilizaremos la cláusula `ORDER BY`, en la que especificaremos los campos por los que deseamos ordenar. Por ejemplo:

```
SELECT `NUM_DEPT`, `NOMBRE`, `SALARIO`
FROM `EMPLEADOS`
ORDER BY `NUM_DEPT`, `SALARIO`;
```

Por defecto, MySQL ordena los valores en orden ascendente. Pero podemos modificar este comportamiento utilizando los modificadores `ASC` o `DESC`:

```
SELECT `NUM_DEPT`, `NOMBRE`, `SALARIO`
FROM `EMPLEADOS`
ORDER BY `NUM_DEPT` DESC, `SALARIO` ASC;
```

5. Consultar varias tablas

En ocasiones, podemos necesitar consultar datos de varias tablas a la vez. Para hacerlo, en primer lugar tendremos que especificar las dos tablas que queremos consultar en la cláusula `FROM`. Luego, en la cláusula `WHERE` especificaremos que queremos mostrar solamente los registros donde coincidan los valores de los campos que relacionan ambas tablas.

Como es probable que los campos que queremos relacionar en las dos tablas tengan nombres idénticos, MySQL nos pide que categoricemos el nombre de dichos campos especificando la tabla de la que provienen, de la siguiente manera: `` `EMPLEADOS`.`NOMBRE` `` (separando el nombre de la tabla del nombre del campo con un punto.

Para mostrar en qué ciudad trabaja cada uno de los empleados de mi empres podría confeccionar una consulta como la siguiente:

```
SELECT `EMPLEADOS`.`NOMBRE`, `DEPARTAMENTOS`.`LUGAR`
FROM `EMPLEADOS`, `DEPARTAMENTOS`
WHERE `EMPLEADOS`.`NUM_DEPT` = `DEPARTAMENTOS`.`NUM_DEPT`;
```